Titulo del Trabajo: Aprendizaje por Refuerzo[1]

Autor1
autor1@um.es

Autor2
autor2@um.es

Autor3
autor3@um.es

6 de marzo de 2026

[1]Comentario que se quiera, si se quiere.

Índice general

Capítulo 1: Problema del Bandido de k-brazos 2

1.1. Introducción 2

1.2. Desarrollo 2

1.3. Algoritmos 3

1.4. Evaluación y Experimentos 4

1.5. Conclusiones 6

Capítulo 1: Problema del Bandido de k-brazos

1.1. Introducción

Descripción del Problema y Relevancia

El problema del bandido de k-brazos (o *Multi-Armed Bandit*) ilustra formalmente el dilema fundamental del Aprendizaje por Refuerzo: el balance iterativo entre la exploración y la explotación. Un agente computacional se enfrenta a un entorno estacionario y estocástico modelado como una máquina tragaperras con k palancas (brazos), cada una ocultando una distribución de probabilidad de recompensa subyacente diferente y desconocida *a priori*.

Con un tiempo límite o presupuesto de T interacciones, la meta del agente es maximizar su recompensa acumulativa. La toma de su decisión es crítica: si confía únicamente en la palanca que le ha concedido mayor ganancia económica hasta ahora (explotación), recae en el riesgo de converger hacia una recompensa local ignorando una palanca numéricamente superior. Si, por el contrario, invierte un esfuerzo desmedido en probar nuevas palancas aleatorias para descubrir el verdadero margen de beneficio (exploración), desperdicia iteraciones cobrando recompensas bajas con la consecuente penalización temporal.

Motivación del Trabajo

La razón principal por la que estudiamos este problema es porque se parece a muchas decisiones prácticas que tomamos en el mundo real (como elegir en qué anuncio invertir dinero o qué tratamiento médico probar). Es un entorno "simple" porque las decisiones que tomamos ahora no cambian las opciones que tendremos en el futuro (cada tirada es independiente). Sin embargo, estudiarlo es clave para entender cómo funciona la inteligencia artificial cuando aprende a base de prueba y error.

Si logramos comprender cómo estos algoritmos básicos (ϵ-Greedy o UCB) interactúan con la duda de *"¿me quedo con lo que ya sé que funciona o arriesgo a probar algo nuevo?"*, habremos construido los pilares necesarios para entender a los verdaderos agentes robóticos complejos que resolverán problemas mucho más difíciles en el futuro.

Objetivos del Informe

El desarrollo de esta primera fase trata de cumplir los siguientes objetivos técnicos: 1. Diseñar y programar un entorno modular en Python capaz de modelar un Bandido bajo tres tipologías de distribución de recompensa continuas y discretas: Normal, Binomial y Bernoulli. 2. Implementar desde cero tres familias clásicas de algoritmos de toma de decisiones: Explotación estocástica (ϵ-Greedy y sus variantes con decaimiento), Ascenso de Gradiente (Softmax y métricas de Preferencia) y Optmismo basado en incertidumbre (UCB-1, UCB-2 y UCB1-Tuned). 3. Evaluar dichas familias sometiéndolas empíricamente frente al entorno mediante $N = 500$ ejecuciones independientes simulando $T = 300$ pasos temporales. 4. Analizar la escalabilidad, la ratio de acierto de selección óptima y la mitigación del arrepentimiento acumulado (*Cumulative Regret*).

Organización del Documento

Para facilitar su asimilación, este primer capítulo estructura su narrativa en el siguiente formato secuencial. En la sección de **Desarrollo** (1.2), se sentarán las bases teóricas definiendo matricial y probabilísticamente las distribuciones a las que se enfrenta el bandido, adjuntando a la par un breve estado del arte de las influencias bibliográficas. Seguidamente, en la sección de **Algoritmos** (1.3), se expondrá la matemática formal y el pseudocódigo subyacente de cada política heurística testeada. Las asunciones y las simulaciones se materializan descriptivamente de la mano de gráficas de rendimiento en la sección referente a la **Evaluación/Experimentos** (1.4). Por último, las métricas son discutidas críticamente en las **Conclusiones** (1.5) para certificar el descubrimiento de los parámetros ganadores globales.

1.2. Desarrollo

Contexto y Antecedentes

El dilema de exploración frente a explotación es una encrucijadas elemental dentro del Aprendizaje por Refuerzo. Mientras que en el aprendizaje supervisado un algoritmo recibe explícitamente la respuesta correcta a modo de etiqueta, en el aprendizaje evaluativo (característico del RL) un agente debe descubrir autónomamente qué acciones brindan la mayor ganancia probándolas. El modelo del *Multi-Armed Bandit* (Bandido Multibrazo) es el paradigma para aislar y estudiar estas propiedades evaluativas y de decisión en un entorno estacionario y no asociativo (es decir, donde el entorno no cambia de estado). Comprender la balanza entre arriesgar ganancias seguras presentes y el potencial de ganancias futuras desconocidas cimienta la matemática que los agentes complejos aplican cuando operan con los valores Q y variables de estado multidimensionales.

Definición Formal del Problema

El entorno estocástico se define bajo el espacio de recompensas discretas o continuas para un conjunto finito de acciones o brazos $a \in \mathcal{A} = \{1, 2, \ldots, k\}$.

El verdadero y desconocido "valor esperado" de elegir una acción a se denota habitualmente como el valor a maximizar $q_*(a)$, y está formulado como la esperanza matemática de la recompensa en un instante t:

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a]$$

A lo largo del presente marco de validación, las recompensas R_t derivan su valor estocástico basándose en tres entornos implementados:

1. **Entorno de Recompensas Normales (Gausianas)** La retribución estocástica de cada acción se muestrea de una distribución diferencial $R_t \sim \mathcal{N}(\mu_a, \sigma_a^2)$, donde la verdadera medida subyacente de recompensa está domi-

nada por su función de densidad de probabilidad (PDF):

$$f(x \mid \mu_a, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu_a)^2}{2\sigma^2}}$$

2. **Entorno de Recompensas Binomiales** En este escenario, la activación del brazo a devuelve una recompensa entera discreta ($x \in \mathbb{N}$) que contabiliza la cantidad total de éxitos sobre n_a ensayos independientes. Es dirigido por la variable de masa probabilística $R_t \sim B(n_a, p_a)$:

$$P(R_t = x) = \binom{n_a}{x} p_a^x (1-p_a)^{n_a - x}$$

3. **Entorno de Recompensas de Bernoulli** Un escenario más sencillo donde el resultado de tirar de la palanca solo puede ser un éxito o un fracaso absoluto ($R_t \in \{0, 1\}$). Modela la variable $R_t \sim Bernoulli(p_a)$ en donde el valor promedio de iteración a estabilizar es exactamente la probabilidad intrínseca al brazo evaluado ($q_*(a) = p_a$):

$$P(R_t) = p_a^{R_t} (1-p_a)^{1-R_t}$$

Enfoque y Justificación Metodológica

Para abordar el problema estocástico, se ha abstraído un *framework* orientado a objetos que desacopla completamente el "Entorno" del "Agente". Esto permite insertar módulos de políticas que basan su decisión algorítmica interactuando con las mismas estructuras pre-configuradas R_t. El presente trabajo se caracteriza por implementar la comparativa simultánea y simétrica del estado del arte (*ϵ-Greedy* frente al Límite de Confianza *UCB* y el cálculo heurístico *Softmax*) bajo un volumen estadístico ($N = 500$ iteraciones promediadas), garantizando un estudio numérico de optimización con mitigación del *Arrepentimiento Acumulado* en ecosistemas tanto lógicos continuos como discretos.

1.2.1. Trabajos Relacionados

El planteamiento algorítmico, su formulación paramétrica y las heurísticas estandarizadas analizadas en este proyecto se fundamentan y se derivan de la literatura existente en el campo del Aprendizaje por Refuerzo: - **Métodos Epsilon y Ascenso de Gradiente (Sutton y Barto, 2018):** El diseño de las estrategias ϵ-Greedy y de los métodos probabilísticos como el *Ascenso de Gradiente* se ha construido siguiendo las reglas y fundamentos explicados en el libro de referencia "Reinforcement Learning: An Introduction" [**?**]. - **Manejo de la Incertidumbre con UCB (Auer et al., 2002):** Para programar los algoritmos UCB (Upper Confidence Bound) de forma robusta y entender cómo utilizan sus fórmulas matemáticas para explorar las opciones menos conocidas, nos hemos apoyado en el estudio "Finite-time Analysis of the Multiarmed Bandit Problem" [**?**].

1.3. Algoritmos

Para resolver el problema del bandido de k-brazos hemos implementado tres enfoques distintos: métodos que combinan explotar lo conocido con explorar al azar, algoritmos que premian probar opciones inciertas para dejar de dudar, y tácticas que eligen las acciones basándose en preferencias relativas en lugar de en ganancias directas. A continuación se justifican sus enfoques y se detalla el funcionamiento lógico de los pilares principales de cada familia algorítmica estudiada.

Familia 1: Lógicas de Exploración-Explotación (ϵ-Greedy)

Justificación: El algoritmo ϵ-Greedy es el punto de partida más clásico en el Aprendizaje por Refuerzo. Su funcionamiento es muy sencillo y rápido de calcular: la mayor parte del tiempo (con probabilidad $1 - \epsilon$) el agente elige la opción que hasta ahora le ha dado más beneficios, pero de vez en cuando (con una pequeña probabilidad ϵ) elige una opción al azar para explorar. A partir de aquí nace una mejora llamada ϵ-Decay, que soluciona el problema de explorar para siempre: va reduciendo poco a poco la probabilidad de explorar (multiplicándola por un factor λ), de modo que, a medida que el agente aprende, se vuelve más seguro y se centra en asegurar las ganancias.

Pseudocódigo de ϵ-Greedy Clásico:

Algorithm 1 Algoritmo ϵ-Greedy para K-Brazos

Require: $\epsilon \in [0, 1]$, Total de acciones K
Inicializar $Q(a) \leftarrow 0$ para todo $a = 1, \ldots, K$
Inicializar $N(a) \leftarrow 0$ para todo $a = 1, \ldots, K$
for cada paso $t = 1, 2, \ldots, T$ **do**
  $p \leftarrow$ Valor aleatorio de Distribución Uniforme(0, 1)
  if $p < \epsilon$ **then**
    $A_t \leftarrow$ Acción aleatoria uniforme en $\{1, \ldots, K\}$
  else
    $A_t \leftarrow \arg\max_a Q(a)$ {Ruptura de empates al azar}
  end if
  Ejecutar A_t, observar recomensa R_t
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}\left[R_t - Q(A_t)\right]$
end for

Familia 2: Optimismo ante la Incertidumbre (UCB)

Justificación: A diferencia de explorar siempre al azar, los métodos de *Upper Confidence Bound* (UCB) son curiosos de forma mucho más inteligente. UCB calcula cuánta confianza tiene en lo que ya sabe de cada palanca. Si una opción se ha probado muy poco, su nivel de incertidumbre es alto, así que el algoritmo decide elegirla para "salir de dudas". Esto convierte a UCB en un método robusto y bueno para no perder el tiempo. En este trabajo hemos estudiado el algoritmo básico UCB1 (que usa un número c para controlar de forma fija esa curiosidad), el modelo UCB2 (que optimiza el proceso probando las opciones en "ráfagas" o bloques cada vez más largos en lugar de recalcular a cada paso), y la variante avanzada UCB1-Tuned (que se ajusta a sí misma midiendo la variabilidad de los premios que recibe).

Pseudocódigo de UCB2:

Familia 3: Ascenso de Gradiente Estocástico (Softmax y Preferencias)

Justificación: Si los algoritmos solo se fijan en los puntos exactos de cada opción (Q), pueden tener problemas para decidir si los premios son muy parecidos (por ejemplo, ganar +85 o +84). Los métodos basados en el Gradiente lo resuelven usando "Preferencias" (H_a). En vez de memorizar valores numéricos rígidos, simplemente aprenden qué opción prefieren en relación al resto y deciden usando porcentajes (con la

Algorithm 2 Algoritmo UCB2

Require: Constante exploratoria $\alpha \in (0, 1)$
Inicializar contador de épocas por brazo: para cada a, $r_a \leftarrow 0$
Iniciar explorando todos los brazos una vez, actualizando su recompensa promedio $Q(a)$
for paso de tiempo $t = K + 1, \ldots$ **do**
  Seleccionar brazo $A_t \leftarrow \arg\max_a \left[Q(a) + \sqrt{\frac{(1+\alpha)\ln(e \cdot t/\tau(r_a))}{2\tau(r_a)}} \right]$ (donde $\tau(r) = \lceil (1+\alpha)^r \rceil$)
  Determinar longitud de la ráfaga: $\Delta \leftarrow \tau(r_{A_t} + 1) - \tau(r_{A_t})$
  for iteraciones $i = 1$ hasta Δ **do**
    Ejecutar A_t, percibir recompensa R
    $N(A_t) \leftarrow N(A_t) + 1$
    $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)} \left[R - Q(A_t) \right]$
  end for
  Actualizar época del brazo: $r_{A_t} \leftarrow r_{A_t} + 1$
end for

Ley Exponencial Suave o Gibbs). En nuestras pruebas, esta estrategia del *Gradiente de Preferencias* trataremos de demostrar si es la que mejor se adapta a los entornos donde las recompensas son simplemente "todo o nada" (como sucede en el modelo de *Bernoulli*).

Pseudocódigo de Base Gibbs / Gradiente:

Algorithm 3 Algoritmo de Variación de Gradiente de Preferencias

Require: Tasa de aprendizaje $\alpha > 0$
Inicializar preferencias $H(a) \leftarrow 0 \ \forall a$
Inicializar recompensa promedio de validación $\bar{R} \leftarrow 0$
for $t = 1, \ldots, T$ **do**
  Calcular probabilidades de la curva Softmax: $P(a) \leftarrow \frac{\exp(H(a))}{\sum_{b=1}^{K} \exp(H(b))} \quad \forall a$
  Elegir acción A_t en base a la distribución de densidad $P(\cdot)$
  Obtener recompensa R_t
  Actualizar la base empírica global $\bar{R} \leftarrow \bar{R} + \frac{1}{t}(R_t - \bar{R})$
  for all $a \in \{1, \ldots, K\}$ **do**
    if $a = A_t$ **then**
      $H(a) \leftarrow H(a) + \alpha(R_t - \bar{R})(1 - P(a))$
    else
      $H(a) \leftarrow H(a) - \alpha(R_t - \bar{R})P(a)$
    end if
  end for
end for

1.4. Evaluación y Experimentos

1.4.1. Configuración Experimental

Para garantizar la validez científica y la total reproducibilidad de los resultados a lo largo del estudio de las tres familias de distribuciones (Normal, Binomial y Bernoulli), el entorno de pruebas fue estandarizado con los siguientes hiperparámetros de simulación global: - **Ejecuciones (*Runs*):** $N = 500$ pruebas independientes estocásticas. Las curvas finales presentadas corresponden a la matriz promediada probabilísticamente sobre este número total de semillas. - **Horizonte de Simulador (*Steps*):** $T = 300$ secuencias temporales de decisión por ejecución. - **Semilla Base Estática:** Se inyectó globalmente una semilla matemática (aleatoria constante, habitualmente `np.random.seed(1024)`) para inicializar la topología generatriz del bandido de forma idéntica ante la comparativa de los tres algoritmos.

Para llevar a cabo las simulaciones, se definieron los siguientes espectros de hiperparámetros iniciales para cada modelo inteligente (Tabla 1):

1.4.2. Métricas de Desempeño

La evaluación del rendimiento integral de los agentes propuestos se analizó multidimensionalmente observando su desempeño iterativo bajo 4 prismas métricos: 1. **Recompensa Promedio Continua (*Average Reward*):** Traza iterativa de la estimación $\bar{R}_t$; refleja la capacidad del agente de percibir y adaptar ganancia bruta promedio. 2. **Ratio Probabilístico de Selección (*% Optimal Action*):** Discrimina de forma binaria el acierto o el fallo explícito al localizar matemáticamente la mejor máquina subyacente $\arg\max q_*(a)$. 3. **Arrepentimiento Acumulado (*Cumulative Regret*):** Mide la acumulación de pérdida teórica u "oportunidad perdida" a nivel temporal de decantarse por opciones penalizadoras frente a un escenario "clarividente". Se busca priorizar gráficas cóncavas y rápidamente estabilizadas (planas). 4. **Histogramas Distribucionales:** Estadísticos puros del muestreo frecuentista con el comportamiento latente final del ecosistema (*Arm Statistics*).

1.4.3. Análisis de Resultados (Gráficas y Comparativas)

La convergencia de rendimiento obtenida tras los experimentos numéricos extraídos de `epsilon_greedy.ipynb`, `ascenso_gradiente.ipynb` y `UCB_algorithms.ipynb` ha posibilitado la categorización de parámetros óptimos para las tres ramas del estado del arte, englobados en la siguiente matriz global (Tabla 2):

Resultados Familia ϵ-Greedy

- **Decaimiento (ϵ-Decay):** Aunque en teoría debería ser mejor, en la práctica el decaimiento no ha supuesto ninguna ventaja clara en los entornos *Normal* y *Binomial*. De hecho, ha funcionado igual o ligeramente **peor** que el modelo básico (ϵ-Greedy puro). De todos los ajustes probados, reducir la probabilidad de explorar progresivamente con un multiplicador $\lambda = 0{,}99$ fue lo más equilibrado. En cambio, si la exploración se reduce muy despacio ($\lambda = 0{,}9999$), el algoritmo pierde demasiado tiempo escogiendo opciones al azar, acumulando muchas pérdidas.
- **Bernoulli**: En este caso particular, hemos experimentado un problema grave: si frenamos la caída de exploración demasiado pronto (fijando un mínimo inamovible de $\epsilon_{min} = 0{,}001$), el algoritmo deja de aprender de golpe. Esto provoca que se estanque prematuramente y no logre igualar la enorme tasa de aciertos que sí consiguen otros competidores.

Tabla 1: Hiperparámetros definidos y comparados por familia algorítmica.

Familia Algorítmica	**Configuración / Algoritmo**	**Hiperparámetros Evaluados**
Exploración-Explotación	ϵ-Greedy	$\epsilon \in \{0{,}0, 0{,}01, 0{,}1\}$
Exploración-Explotación	ϵ-Decay	Tasa de decaimiento $\lambda \in \{0{,}8, 0{,}99, 0{,}9999\}$
Ascenso de Gradiente	Softmax	Temperatura $\tau \in \{0{,}5, 1{,}0, 2{,}0\}$
Ascenso de Gradiente	Gradiente de Preferencias	Tasa de aprendizaje $\alpha \in \{0{,}5, 1{,}0, 2{,}0\}$
Límite Sup. Condianza	UCB1	Constante de exploración $c \in \{0{,}5, 1{,}0, 1{,}5\}$
Límite Sup. Condianza	UCB2	Expansión de confianza temporal $\alpha \in \{0{,}25, 0{,}5, 0{,}75\}$
Límite Sup. Condianza	UCB1-Tuned	Ninguno (Varianza de la Recompensa empírica)

Tabla 2: Parámetros con mayor rendimiento cruzados por distribución.

Familia	**Mejor Hiperparámetro**	**Análisis Crítico**
ϵ-Greedy	$\epsilon = 0{,}1$	Globalmente superior a versiones estáticas conservadoras ($\epsilon = 0{,}01$).
Decaimento	$\lambda = 0{,}99$	Combina convergencia exponencial de exploración. Cae veloz frente a entornos como Bernoulli si su suelo (ϵ_{min}) es severo.
Softmax	Temperatura $\tau \in [0{,}5, 1{,}0]$	Curva modesta en continuas; catastrófico (20 % acierto) en dominios Bernoulli.
P. Gradiente	Ratio de Aprendizaje $\alpha \in [0{,}5, 1{,}0]$	La revelación. Su matriz de distancias absolutas mitiga por completo entornos acotados y discretos (Bernoulli).
UCB	$c = 1{,}0$ (o $c = 0{,}5$ Bernoulli)	Curvas de convergencia teórica muy seguras y logarítmicas. Rinde maravillosamente para incertidumbres Gausianas.
UCB1-Tuned	Ninguno (No paramétrico)	Defensa inquebrantable en todas las distribuciones; el entorno escala su nivel de exploración por la varianza natamente.

Resultados Familia de Ascenso de Gradiente

- **Softmax frente a los entornos:** Mientras que el método Softmax se defiende de forma aceptable en entornos donde los premios varían de forma continua (distribución *Normal*), fracasa completamente cuando los premios son de "todo o nada" (*Bernoulli*). En este último caso, el algoritmo se atasca rápidamente, eligiendo la mejor opción apenas un 20 % de las veces y acumulando muchísimas pérdidas a lo largo del tiempo.
- **Gradiente de Preferencias vs Softmax:** Este ha sido uno de los grandes descubrimientos de las pruebas. El *Gradiente de Preferencias*, al guiarse por "cuánto le gusta" una opción (porcentajes de preferencia) en lugar de intentar memorizar valores exactos de puntos, **ha superado con mucha claridad a Softmax en todos los escenarios**. Su éxito es especialmente espectacular en el entorno *Bernoulli*, donde consigue elegir la mejor palanca casi el 100 % de las veces desde muy temprano. Esto hace que sus pérdidas (arrepentimiento) se queden en unos ridículos 3 puntos tras 300 intentos (frente a los 70 puntos perdidos por Softmax).

Resultados Familia UCB

- **UCB1 vs. UCB2:** El modelo clásico UCB1 aprende bastante bien en casi todas las situaciones probadas, sobre todo cuando su nivel de confianza inicial está fijado en $c = 1{,}0$ (para los modelos *Normal* y *Binomial*). Sin embargo, en el juego de premios binarios (*Bernoulli*) le va mejor si arranca más precavido con un $c = 0{,}5$. Su hermano mayor, **UCB2**, muestra dos caras opuestas: **falla estrepitosamente en los entornos de Binomial y Bernoulli**, donde se atasca eligiendo constantemente la peor opción sin aprender casi nada, pero a cambio resulta ser rapidísimo y el mejor de todos cuando los premios son más naturales y variados (entornos *Normales*).
- **UCB1-Tuned**: Esta versión con "piloto automático" consiguió igualar algunos de los buenos resultados de UCB1 sin necesidad de configurar ningún parámetro a mano. Aún así, sigue pasándolo bastante mal cuando las reglas del juego son drásticas (como en el entorno *Bernoulli*), donde necesita atascarse al menos 100 intentos seguidos antes de lograr darse cuenta de cuál es el brazo ganador.

Para ver todo esto de forma mucho más clara, las siguientes gráficas (sacadas de probar los algoritmos en entornos de premios de tipo *Normal*) muestran un resumen visual perfecto. Tal y como descubrimos al analizar a fondo las simulaciones, **la gráfica del Arrepentimiento Acumulado resulta ser siempre la más útil y fácil de entender**. Esto se debe a que, a diferencia de otras medidas, en el Arrepentimiento se puede distinguir a simple vista si un algoritmo sigue aprendiendo y reduciendo sus pérdidas o si ya se ha estancado por completo.

En la Figura 1, se discierne claramente cómo una caída suave del decaimiento temporal ($\lambda = 0{,}99$) permite achatar asintóticamente el *regret* mitigando estancamientos lineales que sí sufrían ramas no decadentes.

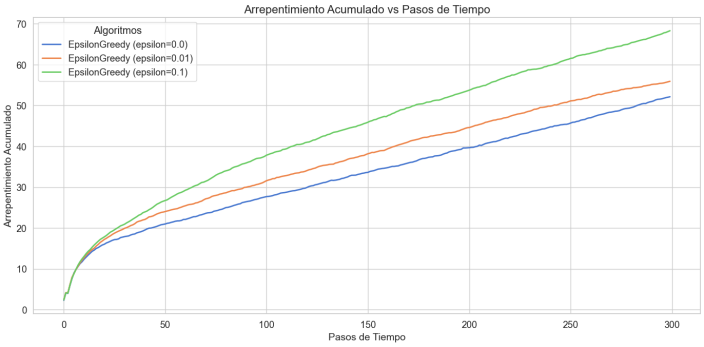

Figura 1: Curva de Arrepentimiento Acumulado para la familia ϵ-Greedy.

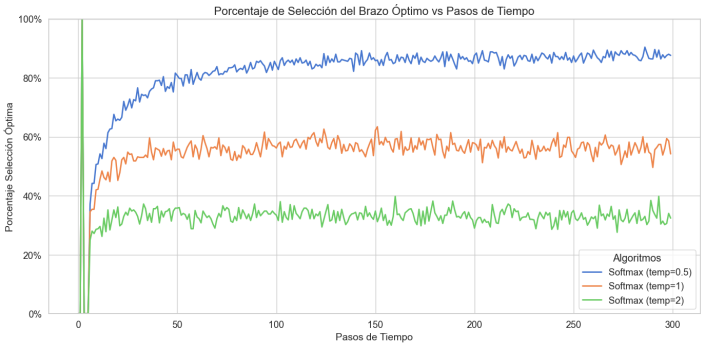

Figura 2: Curva de Arrepentimiento Acumulado comparando Preferencias vs Softmax.

Por otro lado, la comparación que vemos en la Figura 2 nos demuestra algo muy interesante: aunque el algoritmo Softmax arranca bien y consigue ganancias rápidamente, basar las decisiones puramente en las **Preferencias** de cada opción frente al resto (la curva verde) resulta ser una estrategia absolutamente imbatible a medio y largo plazo siempre que se tenga configurada la tasa de aprendizaje correcta ($\alpha = 1$).

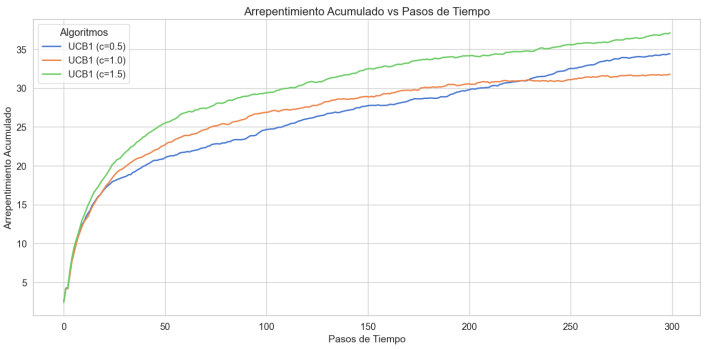

Figura 3: Curva de Arrepentimiento Acumulado demostrando el inquebrantable éxito de UCB1.

Por último, la Figura 3 demuestra en la práctica lo bien que funciona la idea principal del método UCB: si castigamos fuertemente a las opciones que ya hemos probado demasiadas veces, conseguimos que el algoritmo frene en seco sus pérdidas (arrepentimiento). Todo esto lo logra con una precisión asombrosa, de forma rápida y sin necesidad de fórmulas ni ajustes complicados, tal y como atestigua el excelente recorrido de la variante con "piloto automático" (*UCB1-Tuned* en la curva verde).

1.5. Conclusiones

Resumen de Principales Resultados

Tras probar a fondo todos los algoritmos en el problema del bandido de k-brazos, sacamos una conclusión clara: no existe un algoritmo perfecto para todo. La mejor estrategia siempre dependerá del tipo de entorno y de cómo se comporten los premios.

- En entornos donde las recompensas varían de forma continua y lógica (como en la distribución *Normal*), métodos como **UCB1** y especialmente **UCB2** aprenden rapidísimo al elegir de forma inteligente qué opciones explorar. Sin embargo, **solo UCB1** logra mantener ese buen nivel en entornos algo más rígidos como el *Binomial*, ya que UCB2 se atasca por completo.
- Cuando nos enfrentamos a escenarios drásticos de "todo o nada" (recompensas *Bernoulli*), casi todos los algoritmos anteriores fallan de forma estrepitosa y acumulan pérdidas. En estos casos tan extremos, el ganador es el **Gradiente de Preferencias**. En vez de obsesionarse con calcular exactamente cuántos puntos da cada opción, este modelo se limita a comparar qué tan buena es una palanca respecto de las demás; gracias a esto, logra dominar el juego y no equivocarse casi nunca ($Regret \approx 0$).
- Para situaciones variadas o cuando no tenemos mucha potencia de cálculo, recurrir a los clásicos métodos semialeatorios (como ϵ-**Decay** o directamente un ϵ fijo de 0,1) resulta ser una estrategia buena.

Limitaciones del Estudio

La limitación principal de los resultados reportados en este documento es que el juego asume que las reglas no cambian, la propiedad de **estacionalidad** del modelo K-Brazos abstraído. En el mundo real, la característica estocástica y las medias de recompensa (μ) de una máquina o variable evolucionan y mutan con el paso del tiempo. Si transladamos estos algoritmos a la vida real, estos algoritmos podrían fracasar estrepitosamente porque se obcecarían en descartar opciones de escenarios que de repente podrían volverse muy rentables.

Además, este marco experimentacional excluye variables de "contexto" (variables climáticas, visuales o previas a la decisión) de cada tirada o instante t, tratando a las palancas bajo el paradigma probabilístico absoluto e idéntico en cada *step*.

Impacto y Reflexión en el Aprendizaje por Refuerzo

Estudiar a fondo estas tres familias de algoritmos del bandido (apostar a lo seguro, investigar las opciones menos conocidas, o guiarse por las preferencias) no es para nada un ejercicio anticuado o puramente teórico. Al contrario: el concepto del *Arrepentimiento* y cómo minimizarlo son los cimientos más básicos que le dicen a cualquier Inteligencia Artificial cómo debe tomar sus decisiones.

Estas fórmulas e ideas matemáticas conforman el motor in-

terno inicial de decisiones de cualquier agente moderno y complejo en el mundo real (como los sistemas de *Deep Reinforcement Learning* de las IA generativas o de los coches autónomos).

Líneas Futuras de Estudio

El siguiente paso natural para avanzar en este campo sería crear simulaciones de **Bandidos No Estacionarios (*Non-stationary Bandits*)**, es decir, enseñar a la Inteligencia Artificial a jugar en entornos donde las reglas y los premios van cambiando con el tiempo. Para lograrlo, habría que dotar a los algoritmos con capacidades de adaptación "olvidadizas": mecanismos matemáticos que les permitan ignorar lecciones antiguas que ya no sirven y centrarse en lo que está ocurriendo ahora mismo.

Más adelante, el objetivo final sería evolucionar hacia **Bandidos Contextuales (*Contextual Bandits*)**. En este escenario, la IA no jugaría a ciegas, sino que recibiría "pistas" del entorno antes de tener que elegir una opción (por ejemplo, saber qué tiempo hace antes de recomendar un tipo de ropa). Esto abre directamente las puertas hacia sistemas de inteligencia artificial mucho más completos y complejos que los que hemos visto hasta ahora.

Bibliografía

[1] Autor, A. (Año). *Título del artículo.* Nombre de la revista, Volumen(Número), páginas.

[2] Autor, B. (Año). *Título del libro.* Editorial.

Listar todas las referencias utilizadas en el trabajo, siguiendo un formato bibliográfico estándar de LaTeX o de alguna asociación (APA, IEEE, etc.). La información obligatoria para que tipo de publicación la puede consultar en https://bibtex.eu/fields/ Se recomienda utilizar un gestor de referencias como BibTeX para facilitar la organización de las citas solo si va a poner mucha bibliografía.